



Chapter 10: Infusing Your Apps with Machine Learning Using WinML

Artificial intelligence is, without a doubt, one of the most relevant topics in the technology ecosystem, which unlocks new and exciting opportunities to automate tasks that, in the past, simply weren't possible, from recognizing objects in a picture or a video feed to understanding a user's behavior.

These scenarios are powered by machine learning, a technology that enables you to train a model (which is stored in a file) with a set of data. For example, you can train a model to recognize a specific object in a picture. Once you have a model, by identifying patterns, the algorithm will try to apply the same logic used during the training to any picture you provide as input.

The cloud is the technology that has enabled such a big growth of machine learning; thanks to its computing power capabilities, you can train a machine learning model with your data in hours instead of days or weeks. Or you can use many already trained models with a platform-as-a-service approach. Think, for example, of Cognitive Services (<https://azure.microsoft.com/en-us/services/cognitive-services/>), a service provided by Azure that you can use to integrate features such as speech recognition, face recognition, or anomaly detection. All these features are exposed through simple REST APIs or a dedicated SDK that you can consume in any type of application. You just need to pass to the API the input data (such as audio speech or an

image) to get back a result already processed through the built-in models.

There are scenarios, however, where the cloud can introduce challenges. Think, for example, of an application that powers an industrial machine and uses a camera to detect anomalies in the pieces that are produced. In this context, you need to analyze a huge volume of pieces very quickly, so the cloud can become a bottleneck; as fast as your connection can be, there will always be a latency. Additionally, the cost can become prohibitive, since you must transfer a huge amount of data.

In this chapter, we're going to learn how to use WinML to solve these challenges through the following topics:

- A brief introduction to ONNX and WinML
- Evaluating a machine learning model in your Windows application
- Training and using your own machine learning model

Let's start!

Technical requirements

The code for the chapter can be found in the GitHub repository here:

<https://github.com/PacktPublishing/Modernizing-Your-Windows-Applications-with-the-Windows-Apps-SDK-and-WinUI/tree/main/Chapter10>

A brief introduction to ONNX and WinML

To solve the challenges we have outlined in the introduction of this chapter, the ecosystem has introduced the concept of Edge computing. Instead of delegating every operation to the cloud, machine learning models are evaluated directly on a local device, without needing an internet connection. The cloud still plays a critical role, but as a companion; for example, if the recognition rate of an anomaly is under a certain threshold, the application might choose to send the video feed to a cloud service for deeper analysis.

Windows 10 has introduced a platform called WinML to enable any Windows device to become part of the Edge ecosystem. Thanks to this feature, you can evaluate machine learning models directly on your device, using the computing power offered by the CPU and GPU of your machine. WinML exposes a set of APIs that you can integrate into your Windows applications so that you can evaluate a model in your application and process the results.

When you start working on Edge scenarios, the main difference compared to a cloud service is that the machine learning model must be hosted directly by your application. **ONNX** (which is the acronym for **Open Neural Network Exchange**) is one of the most popular standards for open machine learning interoperability. When you have a model based on ONNX, in fact, you can consume it with multiple languages and application types.

There are multiple ways to get access to an ONNX model:

- You can download an existing one published on the internet. In this chapter, we're going to build a Windows application based on WinML starting from an existing model.
- Many of the services and tools dedicated to training machine learning models give you the option to export them to ONNX. In this chapter, we're going to see this scenario first-hand with Azure Custom Vision.

- You can use tools such as ONNXMLTools (available at <https://docs.microsoft.com/en-us/windows/ai/windows-ml/onnxmltools>) to convert existing models created with other machine learning platforms, such as TensorFlow, Keras, or Apple Core ML.

An essential tool to work with ONNX is **Netron**, an open source application available at <https://netron.app/> that you can use to explore different types of machine learning models, including ONNX. The application requires a good knowledge of how machine learning algorithms work, but it's also useful for the scenarios we're going to explore in this chapter to determine the input and output we need to manipulate in the application. Netron can be installed on your machine as a desktop application or used directly from your browser through the web application.

ONNX has evolved over time and models can target a specific version of the standard. It's important to always verify the version before using a model, since it might not be supported by the Windows version you're targeting. For example, the latest version of ONNX, 1.7, is only supported by Windows 11. You can use the table available at <https://docs.microsoft.com/en-us/windows/ai/windows-ml/onnx-versions> as a reference to understand the compatibility level.

Once you have an ONNX model, you can integrate it into your application and, through the WinML APIs, you can pass the input data to perform an evaluation. WinML can be consumed in the following two ways:

- **Natively from the operating system:** Windows ships with a built-in version of WinML that doesn't require any dependency other than enabling your application to support Windows 10 APIs, as we learned in [Chapter 8](#), *Integrating Your Application with the Windows Ecosystem*. This approach is the most flexible one since it doesn't require you to add any extra components to your applica-

tion; additionally, updates and new features are automatically delivered when a new Windows version ships. You will need at least Windows 10, version 1809, to support this approach.

- **From a NuGet package:** In this case, the WinML runtime is directly packaged in your application, which expands the support for this platform back to Windows 8.1. With this approach, however, the size of your application grows (since you must include the runtime as part of the package), and you must take care of updating it when you want to use new features. The package to install is called **Microsoft.AI.MachineLearning**.

Regardless of the approach you choose, what makes WinML powerful is the deep connection with the operating system and the device features. WinML, in fact, abstracts the hardware by enabling developers to manually target the CPU or GPU or to automatically choose the best approach. WinML supports hardware acceleration to provide better performance, by using DirectML, which is a low-level set of APIs that is part of the DirectX family.

Let's start working with WinML now, by building a sample application that can locally evaluate an ONNX model. The features we're going to implement work in the same way regardless of whether you choose to use the native integration or the NuGet package.

Evaluating an ONNX model with WinML

In this section, we're going to start with an existing ONNX model. On the internet, in fact, you can find many pretrained models for some common scenarios, such as object detection or face analysis. A good and reliable source for these models is supported directly by the ONNX community, which curates an official repository on GitHub at <https://github.com/onnx/models>.

The model we're going to use in our application is for object recognition and it's called **SqueezeNet**. Given an image, it's able to recognize objects from 1,000 different categories. It's a great fit for edge scenarios since it provides reliable results with compact size; the model, in fact, weighs only 4 MB, leading to a very small increase in the size of the application. We can download the model directly from the URL <https://github.com/onnx/models/blob/master/vision/classification/squeezenet/model/squeezenet1.1-7.onnx>.

Thanks to this model, we're going to build an application that will enable users to load an image and get back a category that describes it. We can use the Netron application we previously downloaded to explore it. As mentioned in the previous section, unless we have a very good knowledge of machine learning algorithms, it won't be easy to decode the model. However, thanks to the application, we can discover two critical pieces of information we're going to need to use the model in WinML: the input and the output. If we choose **View | Properties** from the top-level menu, Netron will open a panel where we can see that the model expects a property called **data** as input, while the output is stored in a property called **squeezenet0_flatten0_reshape0**.

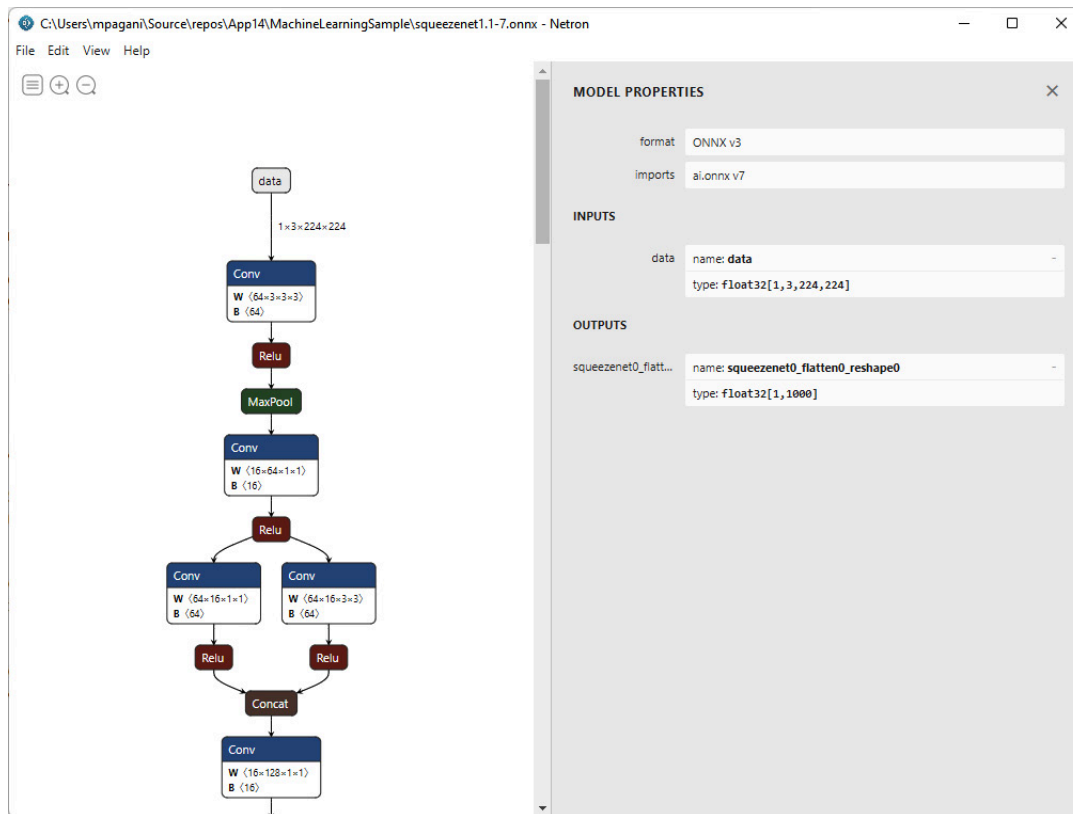


Figure 10.1 – The properties of an ONNX model

We can also see that the output type is an array of numbers, from **1** to **1000**. The model will return, in fact, a probability number for each of the 1,000 categories supported by the model. The category with the highest probability is the one that best describes the content of the image. The category labels, however, aren't included as part of the model. Thanks to Netron, we can, in fact, see that the output is numbers that identify the categories. As such, we also need the label that describes them that we can use to link the identifiers with the category name. In the case of the model we have chosen, this file is published at

<https://github.com/onnx/models/blob/master/vision/classification/synset.txt>. Download this file on your computer, as we're going to need it later.

Now that we have everything we need, we can start building our Windows application. In this chapter, we're going to create a WinUI application, but the same APIs can also be used from a WPF or Windows Forms application. Just remember, as we learned in

Chapter 8, *Integrating Your Application with the Windows Ecosystem*, that if you opt in for WPF or Windows Forms, you must change the **TargetFramework** property of your project to use one of the specific Windows 10 workloads (such as **net6.0-windows10.0.19041.0**).

The starting point of our journey will be the **Blank App, Packaged (WinUI in Desktop)** template in Visual Studio.

Once you have created a new project, the first step is to import the two files we previously downloaded from GitHub: the ONNX model and the text file with the labels. Make sure that, in the **.csproj** definition, these two files are included with **Content** as **Build Action**, as in the following sample:

```
<ItemGroup>
  <Content Include="sysnet.txt" />
  <Content Include="squeezenet1.1-7.onnx" />
</ItemGroup>
```

Now we're ready to start writing some code. Let's see, section by section, all the tasks we need to perform to integrate WinML.

Loading the ONNX model

The first action to take is to load the ONNX model using the WinML APIs so that we can start working with it. A model is represented by the **LearningModel** class, which belongs to the **Windows.AI.MachineLearning** namespace. This class exposes a few static methods that you can use to load a model, which support both unpackaged and packaged scenarios. Let's consider the following sample:

```
private async Task<LearningModelSession>
    CreateLearningSession()
{
    var modelFile = await
```



```
StorageFile.GetFileFromApplicationUriAsync(new
    Uri("ms-appx:///squeezenet1.1-7.onnx"));
    LearningModel model = await
    LearningModel.LoadFromStorageFileAsync(modelFile)
;
    LearningModelSession session = new
    LearningModelSession(model);
    return session;
}
```

In our scenario, we're building a packaged application with WinUI, so we can use the application URI approach we learned how to use in [*Chapter 8, Integrating Your Application with the Windows Ecosystem*](#). Since the model is part of our package, we can load it using the **GetFileFromApplicationUriAsync()** method exposed by the **StorageFile** class, passing as a parameter the URL **ms-appx:///squeezenet1.1.-7.onnx**. We get back a **StorageFile** object, which we can pass to the **LoadFromStorageFileAsync()** static method exposed by the **LearningModel** class.

In the case of an unpackaged app, the **LearningModel** class also exposes methods that support traditional file paths, such as **LoadFromFilePath()**:

```
LearningModel model = LearningModel.LoadFromFilePath
    (@\"C:\model.onnx\");
```

Once we have a model, we can create an evaluation session, by creating a new **LearningModelSession** object, passing as a parameter the model we have just loaded. By default, Windows will automatically create a session on the best available device. However, you can force the session to run on a specific device, by passing a **LearningModelDevice** object as a second parameter. The following sample shows how to force the execution on a GPU powered by DirectX:

```
var session = new LearningModelSession(model, new
```

```
LearningModelDevice(LearningModelDeviceKind.DirectX))  
;
```

Now that we have the model, we also need the labels for the various categories.

Loading the labels

As we mentioned in the introduction, the model we have chosen doesn't include the labels of the categories it supports, but we must load them from an external file we have downloaded from the same GitHub repository. This file (which we have already included in the project) is just a plain text file: each line contains the category index, followed by its description, separated by a comma. The following snippet of code parses the text file to import it into a collection of the **Dictionary<long, string>** type:

```
private async Task<Dictionary<long, string>>  
LoadLabels()  
{  
    var file = await StorageFile  
.GetFileFromApplicationUriAsync(new Uri(  
    "ms-appx:///sysnet.txt"));  
    var text = await FileIO.ReadTextAsync(file);  
    var labels = new Dictionary<long, string>();  
    var records = text.Split(Environment.NewLine);  
    foreach (var record in records)  
    {  
        var fields = record.Split(",", 2);  
        if (fields.Length == 2)  
        {  
            {  
                var index =  
long.Parse(fields[0]);  
                labels[index] = fields[1];  
            }  
        }  
    }  
}
```

```
        return labels;  
    }
```

Also, in this scenario, since we're working with a WinUI-packaged application, we can load the file using an application URI, which is **ms-appx:///sysnet.txt** (since we have placed the text file at the root of the project). The code uses the storage APIs we learned about in [*Chapter 8, Integrating Your Application with the Windows Ecosystem*](#), to read the entire text in memory, and then, for each line, it adds a new item to the dictionary called **labels**; the item key will be the category identifier, while the item value will be the category description.

Now that we have all the requirements ready, we can feed the model with the image we want to recognize.

Preparing the model input

Our model requires, as input, an image, which will then be evaluated against the categories supported by the model. As such, as the first step before performing the evaluation, we must enable the user to load an image. In our sample, we're going to allow users to load images from their computer, by using the **FileOpenPicker** class we learned how to use in [*Chapter 8, Integrating Your Application with the Windows Ecosystem*](#). Let's see the full code snippet first:

```
private async Task<ImageFeatureValue> PickImageAsync()  
{  
    var filePicker = new FileOpenPicker();  
    // Get the current window's HWND by passing in  
the  
    Window object  
    var hwnd = WinRT.Interop.WindowNative  
.GetWindowHandle(this);  
    // Associate the HWND with the file picker  
    WinRT.Interop.InitializeWithWindow  
.Initialize(filePicker, hwnd);
```

```
        SoftwareBitmap softwareBitmap;  
        filePicker.FileTypeFilter.Add("*");  
        var file = await  
filePicker.PickSingleFileAsync();  
        using (IRandomAccessStream accessStream = await  
file.OpenAsync(FileAccessMode.Read))  
        {  
            Windows.Graphics.Imaging.BitmapDecoder  
            bitmapDecoder = await Windows.Graphics  
            .Imaging.BitmapDecoder.CreateAsync(accessStre  
am);  
            softwareBitmap = await bitmapDecoder  
            .GetSoftwareBitmapAsync();  
            softwareBitmap = SoftwareBitmap.Convert  
            (softwareBitmap, BitmapPixelFormat.Bgra8,  
            BitmapAlphaMode.Premultiplied);  
        }  
        VideoFrame input =  
VideoFrame.CreateWithSoftwareBitmap  
        (softwareBitmap);  
        ImageFeatureValue image =  
ImageFeatureValue.CreateFromVideoFrame(input);  
        return image;  
    }  
}
```

The first part of the code should be familiar. It's the solution we learned how to use in [*Chapter 8, Integrating Your Application with the Windows Ecosystem*](#), which enables the usage of the **FileOpenPicker** class in a Win32 application; the solution requires us to get the native handle (HWND) of the application's window and use it to initialize the picker object. Just remember that this sample is specific for a WinUI application. If you're working with WPF or Windows Forms, you'll need to retrieve the HWND using the platform-specific APIs we have seen in previous chapters.

Once we have a reference to the file selected by the user, we open its content through a stream of the **IRandomAccessStream** type. The next steps are needed to prepare the image in the right format expected by the ONNX model, which is BGRA8. It's a 32-bit-per-pixel format in which the blue, green, and red color components are placed before the alpha value. To obtain the image in this format, we use the **BitmapDecoder** class to convert the image into a **SoftwareBitmap** representation, passing **BitmapPixelFormat.Bgra8** as the format type. Once we have a **SoftwareBitmap** object, we can use it to create a **VideoFrame** object (which is a class offered by the **Windows.Media** namespace) thanks to the **CreateWithSoftwareBitmap()** method. Why do we need to perform another conversion? Because the **Windows.AI.MachineLearning** namespace provides a special type to handle images called **ImageFeatureValue**, which can be generated starting from a **VideoFrame** object.

We had to go through a few steps, but finally, now we have the proper input for our model: an image in BGRA8 format with the proper encoding.

Performing the evaluation

Now we have everything we need to perform the evaluation:

- A model, loaded in memory
- A list of labels for the various categories supported by the model
- An image to use as input for the classification

To perform this step, we need some help from the Netron application we installed at the beginning of this chapter. If you remember, by opening our model with this application, we were able to determine the input and output format. We're going to use this information in the next code snippet:

```
private async Task<IReadOnlyList<float>>
```

```
EvaluateModelAsync(LearningModelSession session,
    ImageFeatureValue image)
{
    LearningModelBinding bind = new
    LearningModelBinding(session);
    bind.Bind("data", image);
    var results = await session.EvaluateAsync(bind,
    "0");
    // Retrieve the results of evaluation
    var resultTensor = results.Outputs
    ["squeeze0_flatten0_reshape0"] as TensorFloat;
    var resultVector =
    resultTensor.GetAsVectorView();
    return resultVector;
}
```

Before performing the evaluation, we must first pass the input image to the model. We do this thanks to the **LearningModelBinding** class, which we initialize using the **LearningModelSession** object we previously created. Now, we can pass the input by calling the **Bind()** method, using the following:

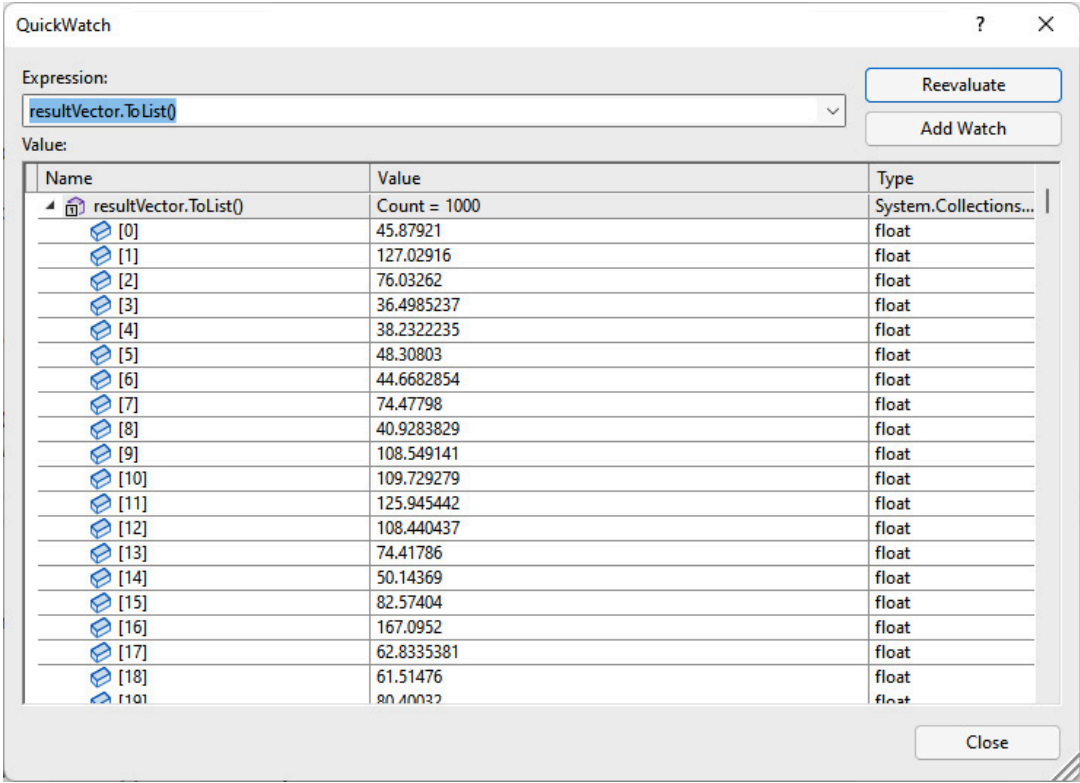
- As the first parameter, the key identifies the input. This is the information we have retrieved from Netron; the expected name of the input is **data**.
- As a second parameter, the **ImageFeatureValue** object we previously created contains our image.

Now, we can finally evaluate the model against our input, by calling the **EvaluateAsync()** method exposed by the **LearningModelSession** object. As parameters, we must pass the **LearningModelBinding** object we just created and an identifier for the execution (we simply pass a fixed string).

When we call this method, WinML will start evaluating the model using the most appropriate technique, based on the hardware the appli-

cation is running on. It might use the GPU, the CPU, or a combination of both. Once the operation is completed, we will find the results in the **Outputs** collection of the result. To decide which result is the correct one, we need to again use Netron. In the panel, we can see that the output is stored inside an item identified by the **squeezenet0_flatten0_reshape0** key.

We get back a **TensorFloat** object, which, by calling the **GetAsVectorView()** method, we can convert into an array of numbers. This result will contain exactly 1,000 items, one for each category supported by the model. For each category (identified by the index in the collection), we will get a number with the calculated probability, as you can see in the following screenshot:



QuickWatch

Expression: `resultVector.ToList()`

Value:

Name	Value	Type
<code>resultVector.ToList()</code>	Count = 1000	System.Collections...
<code>[0]</code>	45.87921	float
<code>[1]</code>	127.02916	float
<code>[2]</code>	76.03262	float
<code>[3]</code>	36.4985237	float
<code>[4]</code>	38.2322235	float
<code>[5]</code>	48.30803	float
<code>[6]</code>	44.6682854	float
<code>[7]</code>	74.47798	float
<code>[8]</code>	40.9283829	float
<code>[9]</code>	108.549141	float
<code>[10]</code>	109.729279	float
<code>[11]</code>	125.945442	float
<code>[12]</code>	108.440437	float
<code>[13]</code>	74.41786	float
<code>[14]</code>	50.14369	float
<code>[15]</code>	82.57404	float
<code>[16]</code>	167.0952	float
<code>[17]</code>	62.8335381	float
<code>[18]</code>	61.51476	float
<code>[19]</code>	80.40032	float

Close

Figure 10.2 – The list of results with the probability calculated by WinML

As the next step, as such, we have to sort the collection so that we can get the items with a higher probability first; they will tell us the category of the object that has been recognized in the image.

Sorting the results

To sort the results, we can rely on the powerful features offered by C# and .NET when it comes to collection manipulation. Here is how we can achieve our goal:

```
private List<(int index, float probability)>
    SortItems(IReadOnlyList<float> resultVector)
{
    // Find the top 3 probabilities
    List<(int index, float probability)>
indexedResults =
    new List<(int, float)>();
    for (int i = 0; i < resultVector.Count; i++)
    {
        indexedResults.Add((index: i,
probability:
        resultVector.ElementAt(i)));
    }
    // Sort the results in order of highest
probability
    indexedResults.Sort((a, b) =>
    {
        if (a.probability < b.probability)
        {
            return 1;
        }
        else if (a.probability > b.probability)
        {
            return -1;
        }
        else
        {
            return 0;
        }
    })
}
```



```

    });
    return indexedResults;
}

```

We have created a new **List** that will store a collection of tuples. Each item will have an index and a probability value. As the first step, we populate the list with all the items that are included in the vector that WinML has generated for us. Then, by using the **Sort()** method, we reorder the collection using the probability as a condition to evaluate. The output of this method will be the same vector we received as input but ordered starting from the item with the highest probability to the one with the lowest probability. This is what the final collection looks like:

Name	Value	Type
indexedResults	Count = 1000	System.Collections...
[0]	(817, 473.8865)	(int, float)
Item1	817	int
Item2	473.8865	float
[1]	(751, 448.814667)	(int, float)
[2]	(436, 444.7551)	(int, float)
[3]	(511, 438.104)	(int, float)
[4]	(479, 437.485016)	(int, float)
[5]	(581, 410.307129)	(int, float)
[6]	(468, 400.940277)	(int, float)
[7]	(717, 392.350677)	(int, float)
[8]	(654, 388.0898)	(int, float)
[9]	(584, 383.902649)	(int, float)
[10]	(477, 376.494)	(int, float)
[11]	(656, 374.6264)	(int, float)
[12]	(518, 367.143616)	(int, float)
[13]	(889, 361.33902)	(int, float)
[14]	(784, 359.392456)	(int, float)
[15]	(407, 356.72168)	(int, float)
[16]	(830, 356.073273)	(int, float)
[17]	(740, 354.302856)	(int, float)

Figure 10.3 – The list of results, this time ordered based on the probability

Each item in the collection has two values: **Item1** is the index of the element in the categories list and **Item2** is the probability that the image is described by that category.

There's a last step we must take care of: we know what the index is, but we don't know what the actual name of the category is. Remember, in fact, that the model returns just the index, not the label. As such, now we must reconcile the list that WinML has generated for us with the list of labels we populated at the beginning starting from the text file.

We can do this very easily. Since the collection of the labels contains the index of the category, we just need to extract the items from the **indexedResults** collection with the same index. This is the code we can use to achieve this goal:

```
for (int i = 0; i < 3; i++)
{
    Console.WriteLine(
        $"{labels[indexedResults[i].index]}\n" with
        probability of
        {indexedResults[i].probability}");
}
```

With this snippet, we are displaying on the console output the top three items in the collection, which means the three categories that have the highest chances of matching the content of the image we have uploaded.

Putting everything together

Now that we have defined all the methods to work with WinML, we can put everything together in a single method that, for example, we can invoke when the user presses a **Button** control in the application. This is what it looks like:

```
private async void OnRecognize(object sender,
    RoutedEventArgs e)
{
    LearningModelSession session = await
```

```

        CreateLearningSession();
        Dictionary<long, string> labels = await
LoadLabels();
        ImageFeatureValue image = await
PickImageAsync();
        IReadOnlyList<float> resultVector= await
EvaluateModelAsync(session, image);
        List<(int index, float probability)>
indexedResults =
SortItems(resultVector);

        // Display the results
        for (int i = 0; i < 3; i++)
        {
            Debug.WriteLine($"\"
{labels[indexedResults[i].
            index]}\\" with confidence of
            {indexedResults[i].probability}");
        }
    }
}

```

This flow recaps everything we have done so far:

1. We have loaded the model starting from the ONNX file and we have created an evaluation session starting from it, using the **LearningModelSession** class offered by WinML.
2. We have loaded the list of labels of the categories supported by the model, starting from a text file provided with the ONNX model.
3. We have enabled users to pick an image from their computer. Then, we have gone through the process of converting the selected file into an **ImageFeatureValue** object, which is the type required as input by WinML.
4. We have run the evaluation, which returns, as a result, a list of the categories and, for each of them, a probability value, which tells us how many chances there are that the content of the image falls into that category.

5. We order the results starting from the one with the highest probability value.
6. Since the model doesn't return, as output, the category labels, we have to manually match the category index returned by the evaluation process with the list that we loaded at the beginning starting from the text file.
7. Finally, we display the first three items in the list, which are the three categories that have the greatest chances of matching the content of the image of the user.

The following screenshot shows what happens when you use WinML to evaluate the ONNX model we downloaded from GitHub, passing, as input, the photo of a car:

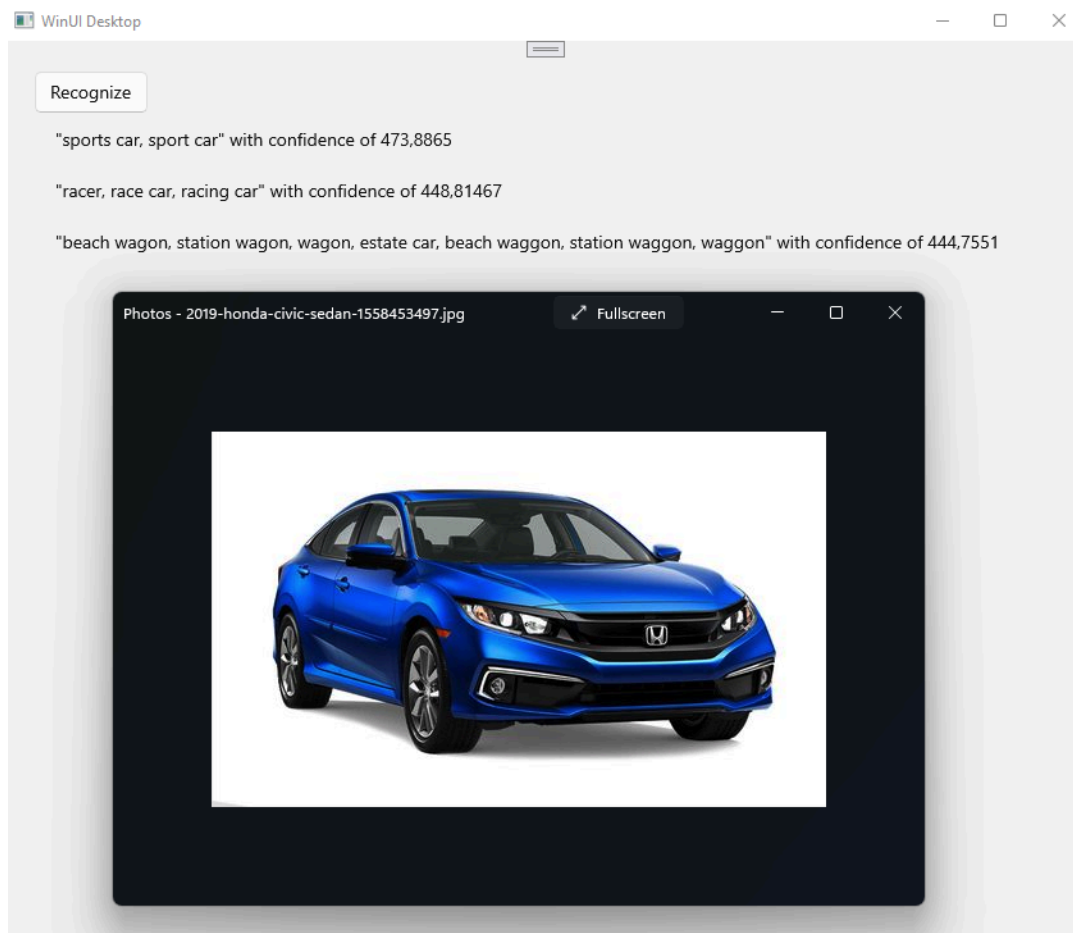


Figure 10.4 – Image of a car properly recognized by the application

You can see how the object recognition works well; the result with the highest probability matches the content of the image (**sport car**), followed by two other categories that are still a good match (**race car** and **station wagon**).

Exciting, isn't it? However, what if we can't find any model in ONNX format that can be a good fit for our scenario? Let's explore how we can train our own one and use it with WinML.

Training and using your own machine learning model

In this section, we're going to reuse the WinML APIs we have learned about so far, but on a different ONNX model. Instead of downloading it from the internet, we're going to create one. Thanks to the power of the cloud, there are many services that we can use to train our models without having to be data scientists. One of these services, provided by Microsoft, is called **Custom Vision**. It's based on Azure, and it requires you to have an Azure subscription. However, the service comes with a free tier, which is a great fit for testing scenarios.

Thanks to Custom Vision, we can train our own model without having to master all the inner details of machine learning algorithms. We will provide a series of pictures and tag them. Then, Custom Vision will train the model based on the information we have provided, enabling it to recognize the objects we have also tagged in every photo, even the ones that aren't part of the original training set. Custom Vision is a great companion for WinML because, among the available options, it supports exporting a trained model to an ONNX file, which we can import into our Windows application.

Let's start by going to <https://www.customvision.ai/> and signing in with a Microsoft account or work account connected to an Azure sub-

scription. If you don't have one, you can create a trial starting from <https://azure.microsoft.com/en-us/offers/ms-azr-0044p/>.

Once you are logged in, click on **New project** and set the following fields:

- **Name:** Choose a name for your project.
- **Description:** This is optional; feel free to leave it empty or fill it with a description of your project.
- **Resource:** Choose the Azure resource that will host the service. If you don't have one, you can choose **Create new** to create a new service, by providing the following:
 - The name.
 - The Azure subscription where you want to host it.
 - The resource group that will contain the service.
 - For **Kind**, choose **CustomVision.Training**.
 - The Azure region that is going to host the service. Choose the one closest to your location.
 - **Pricing tier:** Choose **F0** if you want to use the free tier.
- **Project Types:** Choose **Classification** since we're going to train a model to recognize the content of an image.
- **Classification Types:** Choose **Multiclass** (single tag per image).
- **Domains:** Select **General (compact) [S1]**. It's important to choose one of the compact models since they are the ones that can be exported as ONNX. Once you choose this option, a new field will appear called **Export Capabilities**. Make sure to keep the default value, which is **Basic** platforms. Other domains can give you better results, but they can be consumed only through REST APIs, so they require an internet connection.

Once you have created a new project, you will have access to the main dashboard of the app, which is split into the following three sections:

- **Training Images:** We're going to use these to provide the training data for the model.

- **Performance:** This will tell us how reliable the generated model is.
- **Predictions:** We can use this to test our model.

Let's start with the training!

Training the model

To train the model, we must provide a set of images that describe the object (or the objects) that we want to recognize. In this example, we're going to build a model that can recognize planes in a photo. By clicking the **Add images** button on the dashboard, the tool will give you the option to look for training images on your computer. In the GitHub repository connected to this chapter, you will find a folder with various plane images; otherwise, you can look for them on the web.

Make sure to provide at least five images; it's the minimum requirement to start the training. Of course, the more images you provide, the better the accuracy will be. Once you have selected the images, the tool will ask you to tag them.

We're going to tag them with the **plane** keyword, as shown in the following screenshot:

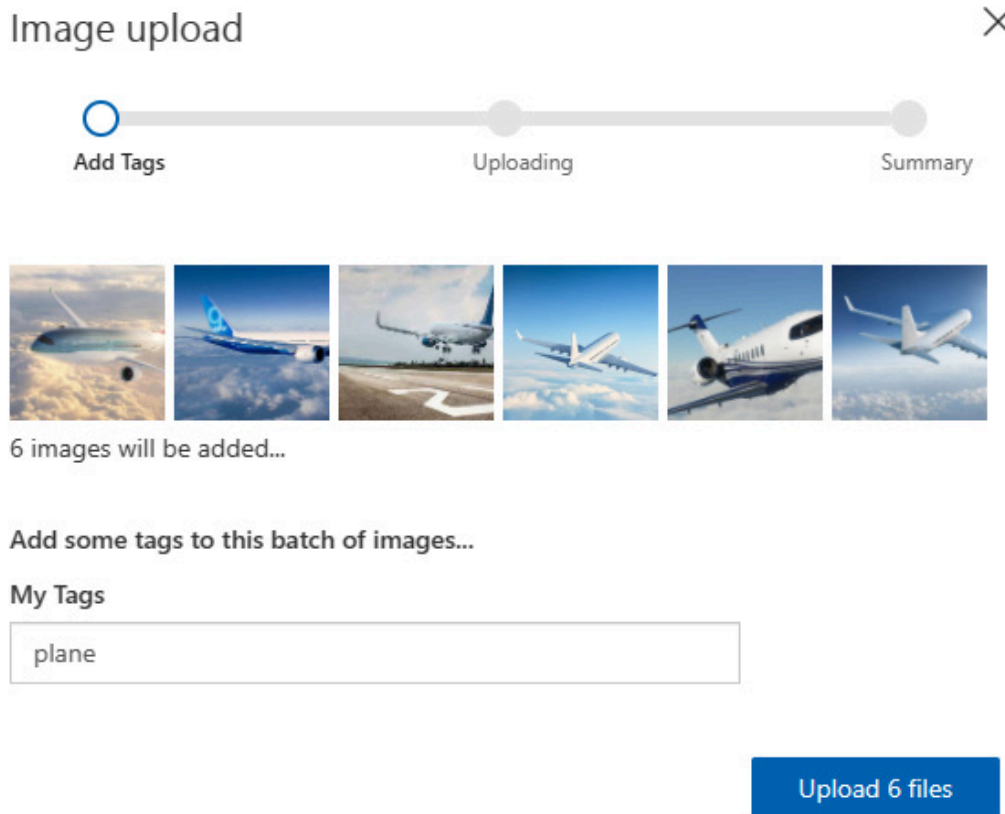


Figure 10.5 – The set of plane images we're providing as the training set for our model

Click on **Upload X files** (where X is the number of images you have uploaded) and wait for the process to finish.

Now, provide a few images that, instead, don't include a plane by again clicking on **Add new images** and, this time, looking for five random pictures you have on your computer, as long as they don't represent a plane. This time, like tags, we're going to use a special one called **Negative**, which means that the image doesn't include any visual subject of any other tag.

In the following screenshot, you can see an example with a set of photos of cars. Of course, I could have used the **car** tag to identify these photos; this way, the final model would be able to recognize both photos of planes and cars. However, for the sake of simplicity, we're going to create a model that is able to recognize only planes, so I'm marking the car photos as **Negative**:

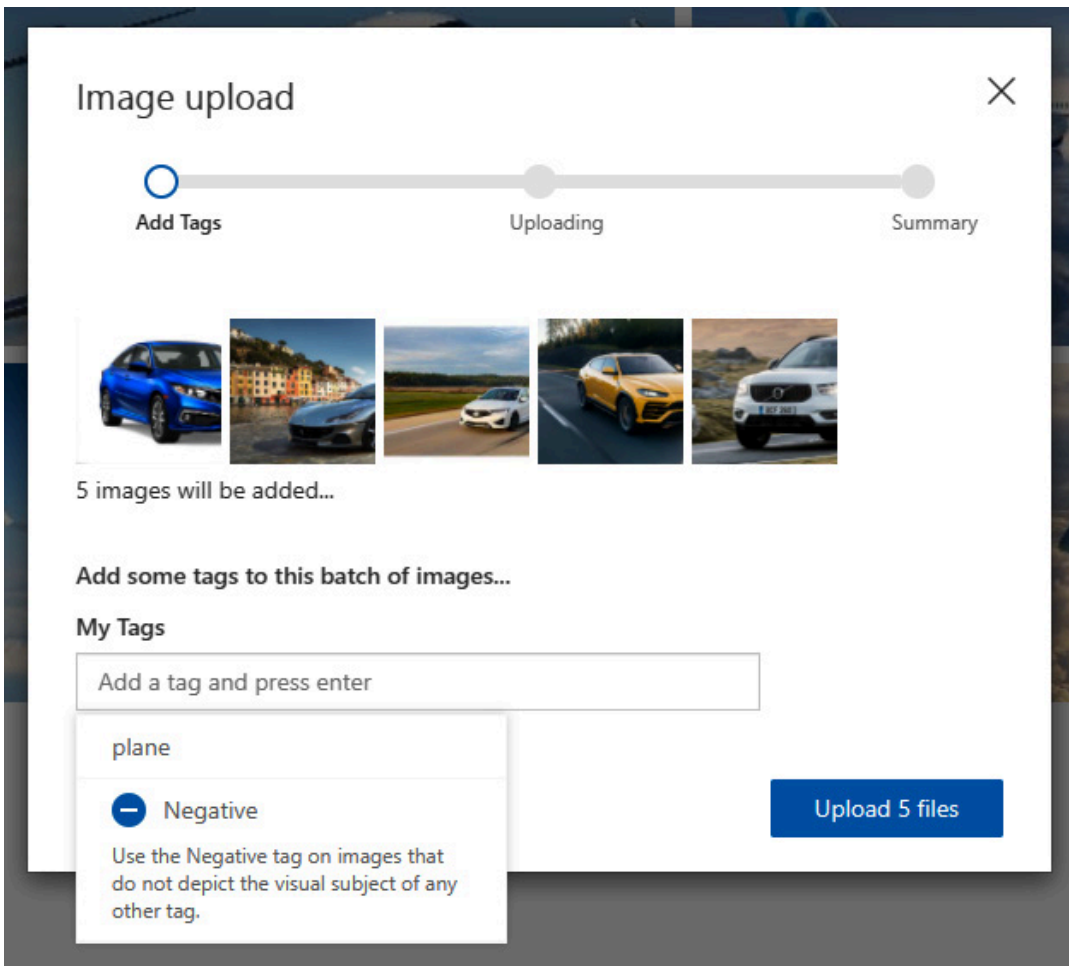


Figure 10.6 – The set of car images is tagged with the special Negative keyword

Now, we have everything we need to start basic training: five images with the content we want to recognize (planes) and five images with different content (cars). Click on the green **Train** button at the top and choose to perform a quick training.

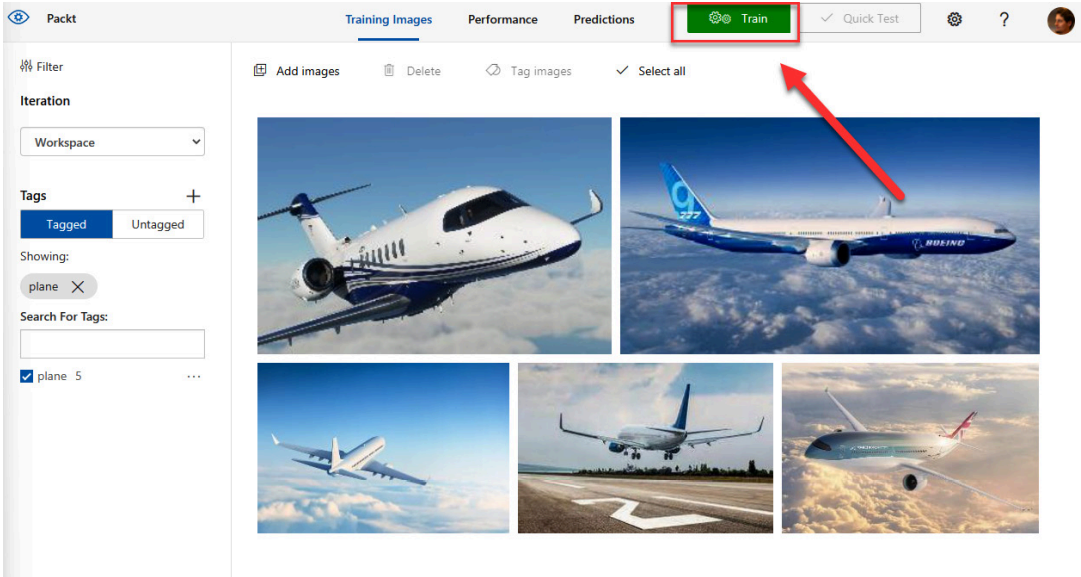


Figure 10.7 – The button to start the training of the model

The training will take a few minutes. The duration of the training is proportional to the number of images you have uploaded. Once the training is completed, the portal will update to show you some statistics about the quality of the model that has been generated.

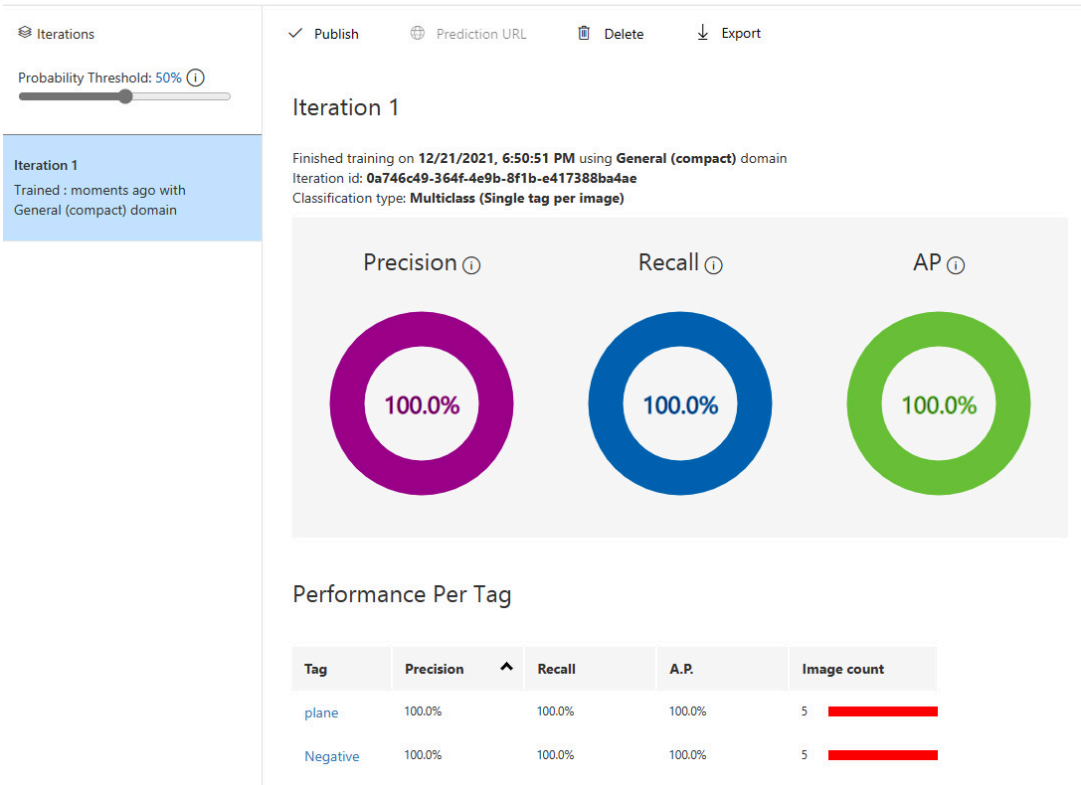


Figure 10.8 – The outcome of the training performed with the provided data

Now that we have a model, we can test it before exporting it thanks to a dedicated feature offered by the Custom Vision portal.

Testing the model

To test the model you have generated, you can use the **Quick Test** button, which is placed near the **Train** button one. You can use this tool to either load an image from the disk or by directly sharing the URL on the internet. Either way, Custom Vision will analyze the image against the model and calculate the probability for each of the tags. In the following screenshot, you can see how the tool has identified that the provided image doesn't contain a plane since **Probability** is **0%**:

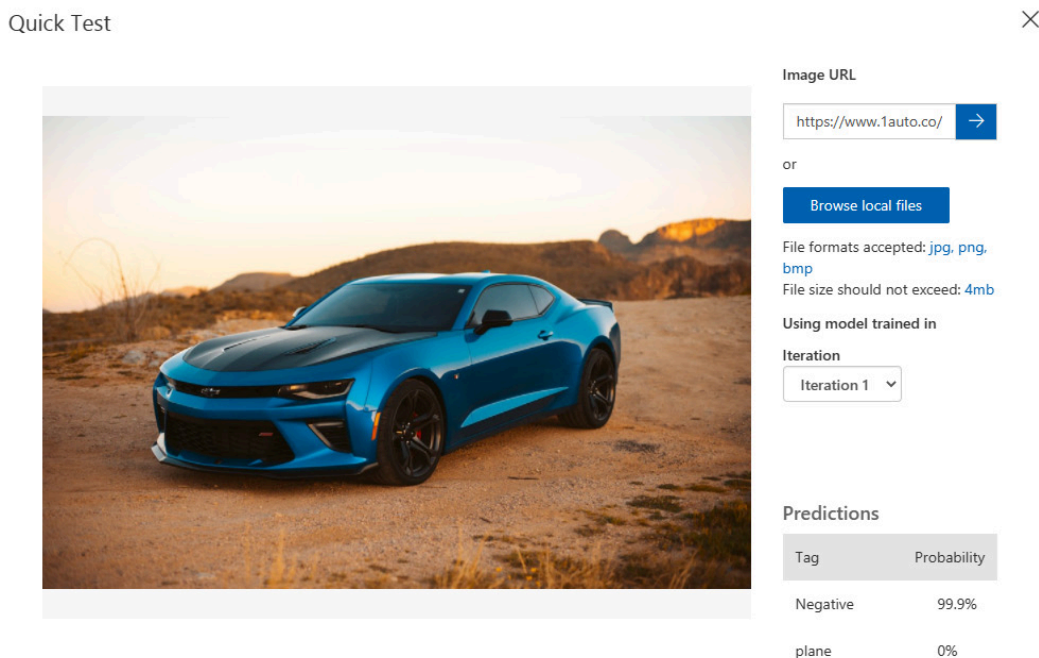


Figure 10.9 – Testing the model with the Quick Test tool

All the images you use for testing will be stored in the **Predictions** section of the portal, which you can use to further improve the model. For example, you can see in the following screenshot that the model doesn't work quite right with the GitHub logo, since it has assigned almost a 90% probability that the image includes a plane. As such, we can assign to the image the **Negative** tag, then press **Save and close**.

The image will be added to the ones used as training data under the **Training Images** section:

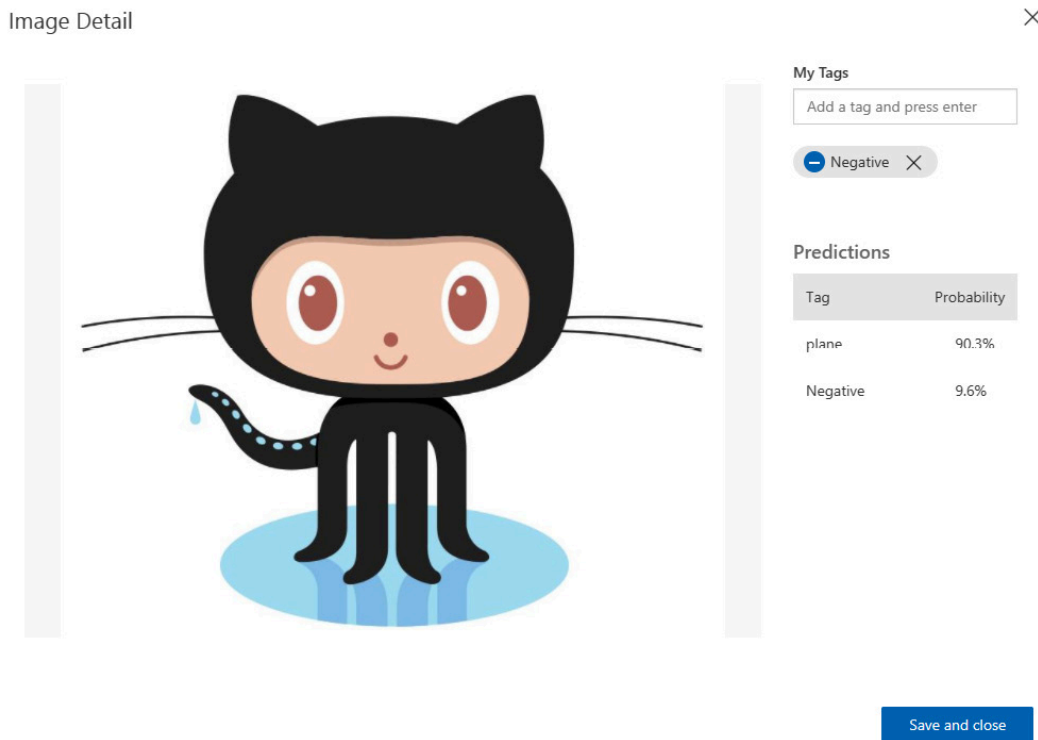


Figure 10.10 – The model didn't properly classify the GitHub logo, so we can fix it by assigning the right tag

Now, if we want, we can again press the **Train** button to run a new iteration, which will take into consideration all the new images that we used for testing the model and, as such, will be more reliable.

Once we are satisfied with the model, we are ready to export it.

Exporting the model

To export the model, you must move to the **Performance** section of the dashboard, where you will find at the top the **Export** button. When you click it, the Custom Vision portal will offer you a series of formats that are supported. Choose **ONNX** and click **Export** to start the process. Once the model has been generated, click on **Download** to get the ONNX file on your local disk.

The model will be included in a ZIP file; so, before using it, extract the file with the **.onnx** extension in a folder. Then, import it into your project in Visual Studio, in the same way you imported the ONNX model you downloaded from GitHub at the beginning of this chapter.

Before testing it, however, we must make a few changes to our application:

- We must open the model with Netron to see whether the inputs and the outputs still match. We can observe that, in both cases, the expected type is the same. The input is an image (represented in BGRA8 format), while the output is an array of numbers. However, this time, the size of the array is only 2, since our model can recognize only two categories: **plane** and **negative**. Additionally, the name of the output is different; it's **model_output** now. As such, we have to change the **EvaluateModelAsync()** method, as follows:

```
private async Task<IReadOnlyList<float>>
    EvaluateModelAsync(LearningModelSession session,
        ImageFeatureValue image)
{
    LearningModelBinding bind = new
        LearningModelBinding(session);
    bind.Bind("data", image);
    var results = await
        session.EvaluateAsync(bind,
            "0");
    // Retrieve the results of evaluation
    var resultTensor = results.Outputs["model_
        output"]
    as TensorFloat;
    var resultVector = resultTensor.
        GetAsVectorView();
    return resultVector;
}
```

- The list of labels we are loading from the text file doesn't match the categories supported by our model anymore. Since, in our case, we support only two categories, we can change the **LoadLabels()** method by manually loading our list of tags in alphabetical order:

```
private Task<Dictionary<long, string>> LoadLabels()
{
    var labels = new Dictionary<long, string>();
    labels.Add(0, "negative");
    labels.Add(1, "plane");
    return Task.FromResult(labels);
}
```

Now, we can run our application and pick any image of a plane. You should get the following result:

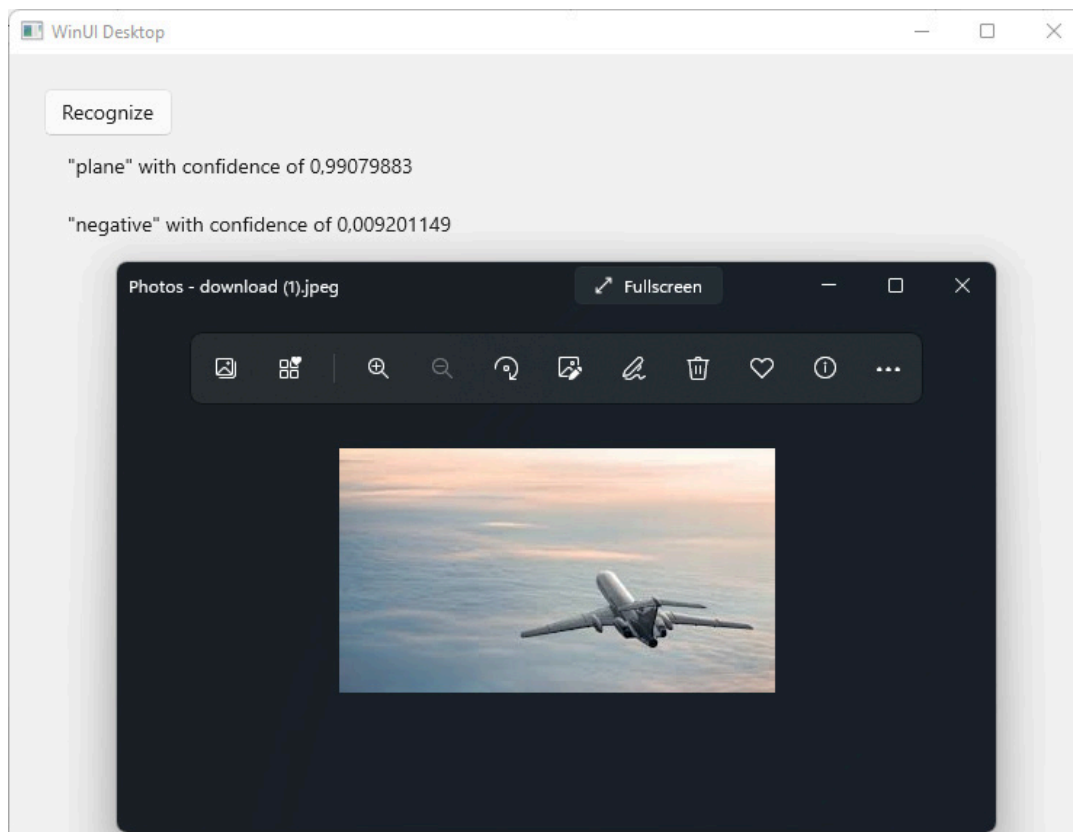


Figure 10.11 – The evaluation result returned by the model we have trained with Custom Vision

As you can see, our model is working. Through the evaluation performed by WinML, our application has recognized that the image is a

photo of a plane.

Custom Vision is just an example of a way to train your model. In the official WinML documentation, available at <https://docs.microsoft.com/en-us/windows/ai/windows-ml/>, you will find many other examples, based on other frameworks that can also train models locally, such as ML.NET or PyTorch.

Summary

In this chapter, we have learned about another great feature provided by Windows, WinML, which enables our computers and tablets to become part of the Edge ecosystem. Thanks to WinML, we can evaluate machine learning models without the need for an internet connection and by using the power of the CPU and GPU of a computer. Object detection, object recognition, and sentiment analysis are just a few of the scenarios that we can enable in situations where we might not have a reliable internet connection, or where we need to process a lot of data in a very short amount of time.

WinML is another example of the new direction of the Windows development ecosystem. It started as a feature that could be used only in Universal Windows Platform applications and it gradually evolved to a platform that can be integrated into any kind of Windows application. In this chapter, we have seen a practical example with a WinUI application, but the same exact code would have worked in a WPF or Windows Forms application, enabling developers to add machine learning to many of the enterprise applications on the market.

With this chapter, we have concluded our modernization journey from the developer aspect of an application. In the remaining chapters, we'll explore another topic that doesn't involve code but is equally important for Windows developers, how to distribute a desktop application, starting in the next chapter.

Questions

1. What is ONNX?
2. WinML uses the power of the cloud to evaluate machine learning models and return the results back to a Windows application. True or false?
3. What is Custom Vision?